

Tema 14

Novedades de C# más destacables



Índice

- Introducción
- Interpolación de cadenas
- Literales de cadena sin formato
- Directivas `global using`
- Espacios de nombres con ámbito de archivo
- Tipos locales de archivo
- `ref struct`
- Palabra clave `required`
- Alias de tipos
- Patrones en listas
- Constructores primarios
- Acceso a índices *from the end*
- Conclusiones y futuro



- Como dijimos en el primer tema, C# es un lenguaje en constante evolución.
- Desde su lanzamiento en el año 2000 como parte de la plataforma .NET, ha ido introduciendo numerosas funcionalidades y ha mejorando el rendimiento considerablemente.

1. Introducción



- Diseñado inicialmente para simplificar la programación orientada a objetos y competir con lenguajes como Java, ha ido incorporando características modernas que lo convierten en una herramienta versátil y poderosa para múltiples paradigmas de programación: funcional, concurrente, e incluso de bajo nivel.

1. Introducción

- En las versiones más recientes, C# ha reforzado su enfoque en:
- **Legibilidad y expresividad del código:** Con mejoras que simplifican la sintaxis y aumentan la claridad del código fuente, como la interpolación de cadenas y los literales de cadena sin formato.
- **Eficiencia y rendimiento:** A través de características como `ref struct` y `ref scoped`, que permiten trabajar directamente con memoria sin comprometer la seguridad.

1. Introducción

- **Organización y productividad:** Nuevas directivas como `global using` y los espacios de nombres con ámbito de archivo eliminan redundancias y hacen que el código sea más limpio y fácil de mantener.
- **Flexibilidad en patrones de diseño:** La incorporación de patrones avanzados en listas y constructores primarios permite expresar estructuras de datos y algoritmos de forma más intuitiva y compacta.

1. Introducción

- Estas innovaciones reflejan un enfoque continuo en equilibrar productividad, seguridad y rendimiento, manteniendo a C# como una de las opciones más atractivas para el desarrollo de aplicaciones modernas.
- A lo largo de este tema, exploraremos las novedades más destacables del lenguaje, con ejemplos prácticos para comprender cómo estas características pueden mejorar la calidad y la eficiencia de nuestro código.

1. Introducción

- La **interpolación de cadenas** es una característica de C# que permite construir cadenas de texto de manera clara y legible al insertar variables o expresiones directamente dentro de un literal de cadena.
- En lugar de usar concatenación con operadores como +, la interpolación mejora la legibilidad y reduce errores, especialmente en cadenas complejas.

2. Interpolación de cadenas

Sintaxis de la interpolación

- Para interpolar una cadena, se utiliza el prefijo \$ antes del literal de cadena, y las expresiones a interpolar se colocan dentro de llaves {}.
- Además de variables simples, se pueden incluir expresiones más complejas, como cálculos, llamados a métodos o incluso condicionales.
- La interpolación puede combinarse con formatos estándar para presentar valores numéricos o fechas.

2. Interpolación de cadenas

Ejemplo

2. Interpolación de cadenas

```
1 int age = 25;
2 string name = "Antonio";
3 string message = $"Hola, me llamo {name} y tengo {
4 Console.WriteLine(message);
```

```
1 double radius = 5.0;
2 double area = Math.PI * Math.Pow(radius, 2);
3 string result = $"El área de un círculo de radio {radius}
4 Console.WriteLine(result);
```

Ventajas de la interpolación de cadenas

- **Claridad y legibilidad:** Evita la complejidad de concatenaciones largas.
- **Menor probabilidad de errores:** Reduce errores al formatear o alinear cadenas.
- **Facilidad de mantenimiento:** Cambiar el contenido de una cadena es más sencillo y menos propenso a introducir fallos.

2. Interpolación de cadenas

- Los **literales de cadena sin formato** son una característica introducida en **C# 11** que simplifica la escritura de cadenas de texto multilínea o cadenas que incluyen caracteres especiales, como comillas dobles o secuencias de escape.
- Este tipo de literal se delimita utilizando tres comillas dobles `"""` en lugar de una, eliminando la necesidad de usar caracteres de escape y conservando el formato de la cadena tal como se escribe en el código fuente.

3. Literales de cadena sin formato

- Los literales sin formato **respetan todos los caracteres** incluidos en el texto, incluidas nuevas líneas y espacios.
- No es necesario usar secuencias de escape para caracteres especiales como `\n`, `\t` o `\"`, por lo que es ideal para cadenas que incluyen comillas dobles o símbolos que normalmente requerirían un carácter de escape.

3. Literales de cadena sin formato

- La **indentación** del texto se ajusta automáticamente para alinearse con el cierre del literal.
- Además, los literales sin formato pueden combinarse con **interpolación de cadenas** para construir textos dinámicamente.
- Para ello, debemos comenzar nuestro literal con `$"""` en vez de `"""`.

3. Literales de cadena sin formato

Ejemplos

3. Literales de cadena sin formato

```
1 string multilineText = ""  
2     Este es un ejemplo de texto multilínea.  
3     Conserva el formato tal y como se escribe,  
4     incluyendo saltos de línea, "espacios" o \n caracter  
5     "";  
6  
7
```

Ejemplos

3. Literales de cadena sin formato

```
1 string interpolatedString = "interpolación de cac
2
3 string multilineText = $"""
4     Si utilizamos el caracter $,
5     También permite la {interpolatedString}.
6     """;
7
8 Console.WriteLine(multilineText);
```

- Las directivas `global using` son una característica introducida en **C# 10** que permite declarar **usings globales**, es decir, directivas de espacio de nombres aplicables a todos los archivos dentro de un proyecto o compilación.
- Esto simplifica el código y reduce la necesidad de repetir `using` en cada archivo, mejorando la limpieza del código y reduciendo la repetición.

4. Directivas `global using`

- Tradicionalmente, cada clase debía especificar los `using` que utilizaba en su código.
- Ahora, si se define un `global using` en alguna clase (lo habitual es utilizar un archivo únicamente para ello, por ejemplo `GlobalUsings.cs`), este `using` estará disponible para el resto de clases del proyecto.

4. Directivas `global using`

- Además, en las últimas versiones de .NET se incluyen una serie de `global usings` por defecto, que podemos ver (y ampliar) en *Propiedades > Usos globales* de nuestro proyecto:

4. Directivas `global using`



Ventajas de las directivas global using

- **Menor repetición:** No es necesario agregar las mismas directivas using en múltiples archivos.
- Esto es particularmente útil en proyectos grandes donde ciertos espacios de nombres se utilizan ampliamente.

4. Directivas global using

- **Código más limpio:** Los archivos individuales de código son más fáciles de leer y mantener al eliminar las directivas redundantes.
- **Consistencia:** Centraliza la configuración de los espacios de nombres utilizados en el proyecto, lo que facilita el mantenimiento.

4. Directivas `global using`

- Los espacios de nombres con ámbito de archivo (introducidos en **C# 10**) simplifican y modernizan la forma en que se definen los espacios de nombres en los archivos de código.
- En lugar de utilizar el enfoque tradicional basado en bloques, puedes declarar un espacio de nombres que abarca **todo el archivo** de manera más concisa.

5. Espacios de nombres con ámbito de archivo

- Tradicionalmente, un espacio de nombres en C# se define utilizando una estructura de bloque, lo que puede agregar una indentación adicional al código.
- Con los **espacios de nombres con ámbito de archivo**, esta misma declaración puede simplificarse eliminando el bloque y reduciendo la indentación.

5. Espacios de nombres con ámbito de archivo

Los espacios de nombres con ámbito de archivo:

- **Aplican a todo el archivo:** Solo puedes tener un espacio de nombres con ámbito de archivo por archivo. Si necesitas declarar múltiples espacios de nombres en el mismo archivo, debes usar el formato tradicional.
- **Mejora estética, no funcional:** No cambia la forma en que funcionan los espacios de nombres; simplemente es una mejora de sintaxis.

5. Espacios de nombres con ámbito de archivo

Ejemplo

- Ambos ejemplos son funcionalmente equivalentes, pero el segundo es más limpio y fácil de leer.

5. Espacios de nombres con ámbito de archivo

```
1 namespace Utilities
2 {
3     public class Calculator
4     {
5         public int Add(int a, int b)
6     }
```

```
1 namespace Utilities;  
2  
3 public class Calculator  
4 {  
5     public int Add(int a, int b)
```

- Los **tipos locales de archivo** (introducidos en **C# 11**) permiten definir tipos específicos para un archivo de código que no están visibles fuera de él.
- Estos tipos ofrecen una manera de encapsular lógica auxiliar o detalles de implementación sin "contaminar" el espacio de nombres global o ser accesibles desde otros archivos.

6. Tipos locales de archivo

- Un tipo local de archivo es cualquier clase, estructura, interfaz o delegado que se declara usando la palabra clave `file` antes de la definición del tipo.
- Esto limita el ámbito de ese tipo al **archivo** donde fue declarado, asegurando que no pueda ser referenciado desde otros archivos en el proyecto.

6. Tipos locales de archivo

Ejemplo

6. Tipos locales de archivo

```
1  file class DiscountCalculator
2  {
3      public static decimal CalculateDiscount(decimal amount,
4      {
5          return amount * (percentage / 100);
6      }
7  }
8
9  public class Order
10 {
11     public decimal TotalAmount { get; set; }
12
13     public decimal GetDiscountedTotal(decimal percentage)
14     {
15         return TotalAmount - DiscountCalculator.CalculateDis
16     }
```

- En el ejemplo anterior, `DiscountCalculator` es un tipo local de archivo y no puede ser usado en otros archivos.
- Esto asegura que la lógica de descuentos permanezca encapsulada en este archivo.

6. Tipos locales de archivo

Vídeo práctico

6. Tipos locales de archivo

Vídeo práctico - código Program

6. Tipos locales de archivo

```
1 namespace ConsoleApp1;
2
3 public class Program
4 {
5     public static void Main()
6     {
7         double number = 2.7689283;
8         int integer = 5;
9
10        string text = $"El decimal es {number:F2}. El
11        Console.WriteLine(text);
12
13        string longText = "Esta es una cadena \nque oc
14        Console.WriteLine(longText);
15
16        string unformattedLiteral =
17            $"""
18            Esta es una cadena
19            que ocupa varias líneas
20            y distinta indentación
21            el decimal es {number:F2}
```

Vídeo práctico - código CalculationsHelper / Class1

6. Tipos locales de archivo

```
1  global using System.Text.Json;
2
3  namespace ConsoleApp1.Helpers;
4
5  internal class CalculationsHelper
6  {
7      public CalculationsHelper()
8      {
9          var aux = new AuxCalculations(
10      }
11 }
12
13 file class AuxCalculations
14 {
15     public int Factor = 7;
16 }
```

- Introducidas en versiones recientes de C#, las características `ref struct` están diseñadas para mejorar el **rendimiento** y la seguridad al trabajar con tipos en el contexto de memoria.
- Estas construcciones son especialmente útiles en escenarios donde la **administración eficiente de memoria** es crucial, como en aplicaciones de alto rendimiento o sistemas con recursos limitados.

7. `ref struct`

ref struct: Estructuras por Referencia

- Una `ref struct` es un tipo de estructura que se asigna en la pila de memoria en lugar del montón (heap), lo que evita la sobrecarga de la recolección de basura (Garbage Collection, GC).
- Este comportamiento hace que las `ref struct` sean ideales para escenarios donde se requiere un acceso rápido y eficiente a la memoria.

7. ref struct

- Debido a su naturaleza, no pueden ser utilizadas como miembros de una clase.
- Tampoco pueden formar parte de una interfaz.

7. ref struct

Ejemplo

- En este ejemplo, `SpanExample` es una `ref struct` que encapsula un `Span<T>`.

7. `ref struct`

```
1  ref struct SpanExample
2  {
3      public Span<int> Numbers { ge
4
5      public SpanExample(Span<int>
```

```
6      {  
7          Numbers = numbers;  
8      }  
9  
10     public void PrintValues()  
11     {  
12         foreach (var number in Nu  
13         {  
14             Console.WriteLine(num  
15         }  
16     }
```

Ventajas de ref struct

1. **Rendimiento mejorado:** La asignación en la pila reduce la carga en el GC.
2. **Seguridad en el manejo de memoria:** Previene el uso de referencias inválidas o fugas de memoria.
3. **Uso eficiente de recursos:** Ideales para operaciones de bajo nivel y en escenarios donde el rendimiento es crítico, como gráficos, procesamiento de datos en tiempo real o juegos.

7. ref struct

Limitaciones

1. **Restricciones estrictas:** No pueden ser usadas en contextos donde puedan escapar del ámbito de la pila (por ejemplo, como parte de una clase).
2. **Sin compatibilidad con interfaces:** No se pueden usar para modelar abstracciones polimórficas, lo que puede limitar su aplicabilidad en ciertos diseños de software.
3. **Curva de aprendizaje:** Entender cuándo y cómo usarlas correctamente puede ser complejo para desarrolladores nuevos en C#.

7. ref struct

- Introducida en C# 11, la palabra clave `required` permite definir propiedades que deben ser **inicializadas obligatoriamente** antes de que se utilice una instancia de una clase.
- Este mecanismo proporciona una forma más explícita y segura de garantizar que ciertas propiedades esenciales no se dejen sin inicializar, evitando errores comunes en tiempo de ejecución.

8. Palabra clave `required`

- Cuando se aplica la palabra clave `required` a una propiedad o campo en una clase, se establece que esa propiedad debe ser asignada durante la **inicialización** de la clase mediante un inicializador de objeto.
- Si una propiedad `required` no se inicializa correctamente, el **compilador** generará un error, ayudando al desarrollador a garantizar la integridad de los objetos.

8. Palabra clave `required`

Ejemplo

- En este ejemplo, Name y Price son required, por lo que debemos inicializarlas al instanciar el objeto, de lo contrario no podremos compilar.

8. Palabra clave required

```
1 public class Product
2 {
3     public required string Name { get; set; }
4     public required double Price { get; set; }
5     public string? Description { get; set; } // Pro
```

```
6    }  
7  
8    // Uso de inicializadores de objeto  
9    Product laptop = new Product  
10   {  
11       Name = "Laptop",  
12       Price = 1000.00,
```

- El uso de la palabra clave `required` es especialmente útil en escenarios donde nos integramos con sistemas externos.
- Por ejemplo, es altamente probable que para realizar una llamada a una API necesitemos pasar usuario y *token*, por lo que modelar esos campos como `required` en nuestra capa puede prevenir futuros errores en la comunicación.

8. Palabra clave `required`

- Un alias de tipo en C# permite definir un nombre alternativo o abreviado para un tipo existente.
- Esto puede ser útil para simplificar el código, mejorar la legibilidad, o manejar conflictos entre nombres de tipos en diferentes espacios de nombres.
- Los alias se crean utilizando la palabra clave `using`.

9. Alias de tipos

- C# incluye alias predefinidos para los tipos del sistema. Por ejemplo:
 - `int` es un alias de `System.Int32`.
 - `string` es un alias de `System.String`.
- Estos alias se utilizan comúnmente en lugar de los nombres de los tipos del sistema por su concisión y claridad.

9. Alias de tipos

- También podemos definir alias personalizados para tipos más complejos mediante la palabra clave `using`.
- Esto es particularmente útil en escenarios como:

9. Alias de tipos

- **Simplificación de nombres largos o complejos:**

9. Alias de tipos

```
1  using CustomerList = System.Collections.Generic.List<Customer>;
2
3  class Program
4  {
5      static void Main()
6      {
7          CustomerList customers = new CustomerList
8          {
9              new Customer { Name = "Antonio Díaz", Age = 35 },
10             new Customer { Name = "María López", Age = 28 },
11          };
12
13          foreach (var customer in customers)
14          {
15              Console.WriteLine($"{customer.Name}, {customer.Age}");
16          }
17      }
```

- **Evitar conflictos de nombres en espacios de nombres:** Si dos bibliotecas tienen tipos con el mismo nombre, puedes usar alias para diferenciarlos:

9. Alias de tipos

```
1 using ProjectAList = ProjectA.Models.List;  
2 using ProjectBList = ProjectB.Models.List;  
3  
4 class Program  
5 {  
6     static void Main()  
7     {  
8         ProjectAList listA = new ProjectAList();  
9         ProjectBList listB = new ProjectBList();  
10    }  
11 }
```

- C# también permite definir alias globales utilizando `global::`.
- Esto es útil cuando deseas asegurarte de referenciar un tipo específico que podría estar oculto por un alias o una definición local:

9. Alias de tipos

```
1  using MyString = global::System.String;
2
3  class Program
4  {
5      static void Main()
6      {
7          MyString text = "Using global alias";
8          Console.WriteLine(text);
9      }
10 }
```

Beneficios de los Alias de Tipos

- **Claridad:** Simplifican nombres largos o genéricos, haciendo que el código sea más legible.
- **Resolución de conflictos:** Permiten diferenciar entre tipos con el mismo nombre de diferentes bibliotecas.
- **Mantenibilidad:** Si un tipo cambia, solo necesitas actualizar el alias, no cada referencia en el código.

9. Alias de tipos

Vídeo práctico

9. Alias de tipos

Vídeo práctico - código Program

9. Alias de tipos

```
1  using IntList = System.Collections.Generic.List<int>;
2  using ProjectBProduct = ConsoleApp1.ProjectB.Product;
3  using ProjectAProduct = ConsoleApp1.ProjectA.Product;
4
5  namespace ConsoleApp1;
6
7  public class Program
8  {
9      public static void Main()
10     {
11         Console.WriteLine("Hello, World!");
12
13         double[] numbers = [2.45664, 0.1207402, -456.
14         DoubleSpan doubleSpan = new DoubleSpan(number
15         doubleSpan.DoCalculations();
16
17         Person person = new Person() { Name = "Pepe",
18
19         IntList numbersList = new IntList();
20
```

Vídeo práctico - código Person

9. Alias de tipos

```
1  internal class Person
2  {
3      public required string Name { get; set; }
4      public int Id { get; set; }
5      public required int Age { get; set; }
6
7      public Person()
8      {
9          Age = 0;
10     }
11 }
```

Vídeo práctico - código Products

9. Alias de tipos

```
1 namespace ConsoleApp1.ProjectA
2 {
3     internal class Product
4     {
5         public string Name { get; set; }
6         public double Price { get; set; }
7     }
8 }
9
10 namespace ConsoleApp1.ProjectB
11 {
12     internal class Product
13     {
14         public string Description { get;
15         public int Quantity { get; set;
16     }
17 }
```

- Con las recientes versiones de C#, los **patrones** han evolucionado para permitir una manipulación más clara y expresiva de colecciones como las listas.
- Estos patrones permiten realizar comprobaciones, desestructuraciones, y otras operaciones complejas de manera sencilla y legible.
- Se usan típicamente con la instrucción `switch`, la expresión `is`, o en combinaciones con `when` para validaciones más detalladas.

10. Patrones en listas

- Los patrones en listas proporcionan formas de:
- **Verificar la estructura de la lista:** Como el tamaño, el contenido, o elementos específicos.
- **Desestructurar listas:** Accediendo a elementos directamente sin la necesidad de índices explícitos.
- **Combinar patrones:** Como patrones de elementos con lógica adicional.

10. Patrones en listas

Tipos Comunes de Patrones en Listas

- **Patrón de Lista Vacía ([])**: Verifica si una lista está vacía.

10. Patrones en listas

```
1 List<int> numbers = new List<int>();  
2  
3 if (numbers is [])  
4 {  
5     Console.WriteLine("La lista está  
6     vacía");  
7 }
```

- **Patrón de Coincidencia Exacta:** Verifica si la lista tiene un tamaño específico y coincide con ciertos valores.

10. Patrones en listas

```
1 List<int> numbers = new List<int> { 1, 2, 3 };
2
3 if (numbers is [1, 2, 3])
4 {
5     Console.WriteLine("La lista coincide exactamente co
6 }
```

- **Patrón de Prefijo (..):** Se utiliza para verificar los elementos iniciales de una lista, ignorando el resto.

10. Patrones en listas

```
1 List<int> numbers = new List<int> { 1, 2, 3,  
2  
3 if (numbers is [1, 2, ..])  
4 {  
5     Console.WriteLine("La lista empieza con  
6 }
```

- **Patrón de Sufijo (..):** Similar al prefijo, pero enfocado en los elementos finales.

10. Patrones en listas

```
1 List<int> numbers = new List<int> { 1, 2, 3, 4 }
2
3 if (numbers is [.., 3, 4])
4 {
5     Console.WriteLine("La lista acaba con ...")
6 }
```

- **Patrón de Intervalo (_ y . .):** Utiliza _ para ignorar elementos específicos y . . para intervalos desconocidos.

10. Patrones en listas

```
1 List<int> numbers = new List<int> { 1,  
2  
3 if (numbers is [1, _, 3, ..])  
4 {  
5     Console.WriteLine("""  
6         La lista comienza con 1,  
7         tiene 3 en el 3er elemento  
8         y puede tener más después.  
9         """);  
- }
```

- **Patrones combinados y variables:** Puedes combinar patrones y también guardar en variables alguno de los elementos de la lista.

10. Patrones en listas

```
1 List<int> numbers = new List<int> { 1, 2, 3, 4 };  
2  
3 if (numbers is [1, 2, .. var rest] && rest.Count > 2)  
4 {  
5     Console.WriteLine("La lista comienza con 1, 2 y tiene más de  
6     ");
```

Ejemplo

- En este ejemplo clasificamos una lista mediante un switch. El primer patrón que se cumpla, será el que se guarde como resultado. En este ejemplo, *"Empieza con 1, 2"*.

10. Patrones en listas

```
1 List<int> numbers = new List<int> { 1, 2
2
3 string result = numbers switch
4 {
5     [] => "Lista vacía",
6     [1, 2, 3] => "La lista es exactamente
7     [1, 2, ..] => "Comienza con 1, 2",
```

```
8  
9  
10 };  
11  
[ .., 3, 4] => "Acaba con 3, 4",  
- => "Otro patrón"
```

Ventajas de Usar Patrones en Listas

1. **Legibilidad:** El código es más declarativo, ya que puedes expresar exactamente lo que estás buscando.
2. **Mantenimiento:** Los patrones hacen que las validaciones y manipulaciones sean más intuitivas y fáciles de cambiar.
3. **Eficiencia:** Combinado con `switch` y expresiones condicionales, puedes evitar código redundante como múltiples bucles o verificaciones manuales.

10. Patrones en listas

- Los **constructores primarios** son una característica introducida en la penúltima versión de C# (12.0) que simplifica la inicialización de objetos al permitir definir un constructor directamente en la declaración de la clase.
- Además, esto elimina también la necesidad de asignar valores a las propiedades en un constructor tradicional.
- De esta manera además unimos la declaración de la clase y su constructor en una sola línea.

11. Constructores primarios

Ejemplo

- En este ejemplo Name y Age son parámetros del constructor primario y los asignamos a sus propiedades sin crear explícitamente el constructor.

11. Constructores primarios

```
1 public class Person(string name, i
2 {
3     public string Name { get; } =
4     public int Age { get; } = age;
5 }
```

- El ejemplo anterior utilizando código "clásico" sería:

11. Constructores primarios

```
1  public class Person
2  {
3      public string Name { get; set; }
4      public int Age { get; set; }
5
6      public Person(string name, int
7      {
8          Name = name;
9          Age = age;
10     }
11 }
```

Ventajas de los Constructores Primarios

- **Concisión:** Elimina la necesidad de escribir constructores y asignaciones repetitivas.
- **Legibilidad:** Centraliza la inicialización y hace el código más claro.
- **Reducción de Errores:** Menos código implica menor probabilidad de errores.

11. Constructores primarios

- El acceso a índices **"from the end"** (desde el final) es una nueva característica de C# que permite acceder a los elementos de una colección utilizando índices negativos.
- Este enfoque simplifica la lectura de los últimos elementos de una colección sin necesidad de calcular manualmente su posición.

12. Acceso a índices *from the end*

Sintaxis

- Para especificar un índice negativo, usamos el operador `^`.
- De esta manera, se cuenta desde el final de la colección el número especificado como índice.

12. Acceso a índices *from the end*

Ejemplo

- En este ejemplo, accedemos al último elemento mediante `[^1]` (en vez del acceso clásico que sería `[rainbowColors.Length - 1]`) y al 4º por la cola mediante `[^4]`.

12. Acceso a índices *from the end*

```
1 string[] rainbowColors = { "Rojo", "Naranja",  
2     "Verde", "Azul", "Añil", "Violeta" };  
3  
4 // Acceso usando índices tradicionales  
5 string firstColor = rainbowColors[0]; // "Roj  
6  
7 // Acceso usando índices "from the end"
```

```
8 string lastColor = rainbowColors[^1]; // "Vio  
8 string secondLastColor = rainbowColors[^4]; //
```

Uso Combinado con Rangos

- El acceso "from the end" se combina perfectamente con la funcionalidad de **rangos** para obtener subrangos de una colección.
- `^2..^0` define un rango que comienza desde el segundo elemento desde el final ("Añil") y termina en el último elemento ("Violeta").

12. Acceso a índices *from the end*

```
1 var lastTwoColors = rainbowColors[^2..^0]; // ["Añil", "Violeta"]
```

Vídeo práctico

12. Acceso a índices *from the end*

Vídeo práctico - código Program

12. Acceso a índices *from the end*

```
1  public static void Main()
2  {
3      int[] numbers = { 4, 124, 42, 10, 2, -5, 1, 2, 3 }
4
5      Console.WriteLine(numbers is [4, 124, 42, 10, 2, -
6      Console.WriteLine(numbers is [4, 124, ..]));
7      Console.WriteLine(numbers is [.., 2, 3]);
8      Console.WriteLine(numbers is [4, .., 3]);
9      Console.WriteLine(numbers is [_, 124, ..]);
10
11     switch (numbers)
12     {
13         case [4, ..]:
14             Console.WriteLine("comienza con 4");
15             break;
16         case [_, _, 42]:
17             Console.WriteLine("El tercer elemento es 4
18             break;
19     }
20
```

Vídeo práctico - código Car

12. Acceso a índices *from the end*

```
1 internal class Car(string brand, string mode
2 {
3     public string Brand { get; set; } = bran
4     public string Model { get; set; } = mode
5     public int Year { get; set; } = year;
6 }
```

- En este curso hemos realizado un repaso a las características más importantes de C# y hemos aprendido las bases de la programación, haciendo especial hincapié en la POO.
- C# es un lenguaje en constante evolución que seguirá introduciendo novedades y mejoras año tras año. Sin embargo, con una base sólida podemos adentrarnos en el mundo de la programación sin miedo.

13. Conclusiones y futuro

Posibilidades al conocer C#

- Con un conocimiento sólido de C#, las oportunidades en el desarrollo de software son amplias y diversas.
- C# es uno de los lenguajes más utilizados y demandados en la actualidad que además es utilizado en ámbitos muy diversos.
- Podremos enfocar nuestra carrera dependiendo de nuestras preferencias.

13. Conclusiones y futuro

Programación Backend

- C# es la base de **ASP.NET Core**, un marco versátil para construir aplicaciones web robustas, seguras y escalables.
- Desde servicios REST hasta aplicaciones en la nube, C# es una elección ideal para desarrolladores backend.

13. Conclusiones y futuro

Desarrollo de Interfaces de Usuario

- Gracias a herramientas como **Windows Presentation Foundation (WPF)** y **WinForms**, C# permite construir interfaces de usuario modernas y funcionales para sistemas de escritorio.
- En la actualidad siguen apareciendo tecnologías basadas en WPF que modernizan y amplían compatibilidad con otros sistemas como **.NET MAUI** o Blazor.

13. Conclusiones y futuro

Aplicaciones Móviles

- Con **Xamarin** y **.NET MAUI**, es posible desarrollar aplicaciones móviles multiplataforma utilizando un solo código base en C#.
- Gracias a estos frameworks, podemos crear la misma aplicación para **Android e iOS**, siendo por ello una plataforma ampliamente utilizada en aplicaciones profesionales.

13. Conclusiones y futuro

Inteligencia Artificial y Videojuegos

- El lenguaje también encuentra aplicación en dominios como **machine learning**, utilizando frameworks como **ML.NET**.
- En cuanto al mundo del desarrollo de videojuegos, en la actualidad es uno de los lenguajes más utilizados gracias al motor **Unity**.

13. Conclusiones y futuro

Continuar con la Programación Avanzada en C#

- Este curso ha proporcionado una introducción integral a C#, cubriendo desde fundamentos hasta temas intermedios.
- Sin embargo, para aquellos interesados en profundizar, se encuentra disponible el **Curso de C# 13 Avanzado**.

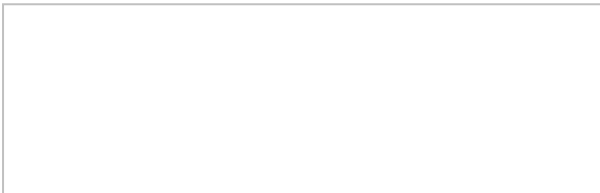
13. Conclusiones y futuro

- En este curso se eleva el nivel de los contenidos, aprendiendo temas más abstractos como los patrones de diseño o la gestión de memoria.
- También se aprenderá a diseñar aplicaciones con interfaz de usuario, integrar librerías externas a nuestros proyectos o realizar tests unitarios.
- Con el curso de **C# 13 Avanzado** seremos capaces de desarrollar aplicaciones "reales".

13. Conclusiones y futuro

Diapositiva final

En este tema hemos repasado alguna de las novedades más útiles de las últimas versiones de C#. También hemos dado unas pinceladas a los siguientes pasos a tomar para continuar trabajando con C#.



Speaker notes